General POINTERS, STRUCTURE, UNIONS

PART - A

MICROVISION EMBEDDED PVT LTD

Q 1. Analyze the following code. Determine what integer value gets printed, or if it triggers undefined behavior/crash, explain precisely why ?

```
#include <stdio.h>
#include <stdint.h>
typedef void (*action_t)(int *);
void add_five(int *val) { *val += 5; }void mul_two(int *val) { *val *= 2; }
int main(void) {
 action_t ops[] = {add_five, mul_two};
   action_t *vptr = ops;
   int x = 10;
   // Complex Pointer Manipulation
   uintptr_t base = (uintptr_t)vptr;
   action_t *p1 = (action_t *)(base + sizeof(action_t));
   (*(p1 - 1))(&x);
   (*p1)(&x);
   printf("x = %d\n", x);
   return 0;
}
```

Q 2. Assuming a 64-bit Little-Endian platform with 8-byte alignment rules, what does this program print ?

```
#include <stdio.h>
#include <stdint.h>
struct RawData {
   uint16_t header;
   uint32_t payload;
   uint8_t  flag;
} __attribute__((packed));
struct HeaderView {
   uint8_t b0;
   uint8_t b1;
};
int main(void) {
```

MICROVISION EMBEDDED PVT LTD

```
    struct RawData data = {0x1234, 0xAABBCCDD, 0xFF};
    uint8_t *ptr = (uint8_t *)&data;
    // Shift pointer to payload
    uint32_t *payload_ptr = (uint32_t *)(ptr + offsetof(struct RawData, payload));
    struct HeaderView *h_ptr = (struct HeaderView *)(ptr + offsetof(struct RawData,
header));

  printf("Payload: 0x%X, Header B0: 0x%X\n", *payload_ptr, h_ptr->b0);
    return 0;
}
```

Q 3. What is the exact printed integer output of the following nested pointer expression?

```
#include <stdio.h>
int main(void) {
    int grid[3][4] = {
        { 1,  2,  3,  4},
        { 5,  6,  7,  8},
        { 9, 10, 11, 12}
    };
    int (*p)[4] = grid;
    int *ptr = (int *)(p + 2);
    printf("%d\n", ptr[-5]);
    return 0;
}
```

Q 4. What is the output of this program?

```
 #include <stdio.h>
int main(void) {
  char *vec[] = {"Alpha", "Beta", "Gamma", "Delta"};
 char **app[] = {vec + 3, vec + 2, vec + 1, vec};
  char ***pp = app;
   printf("%s ", **++pp);
   printf("%s ", *--*++pp + 3);
  printf("%s\n", pp[-1][1]);
  return 0;
}
```

MICROVISION EMBEDDED PVT LTD

Q 5. What does this type-punning pointer snippet output on a standard system where int is 4 bytes and short is 2 bytes (Little-Endian)?

```
#include <stdio.h>
#include <stdint.h>
typedef union {
   uint32_t word;
   uint16_t half[2];
   uint8_t  bytes[4];
} Register;
int main(void) {
   Register reg = {.word = 0x12345678};
   uint16_t *h_ptr = &reg.half[1];
   uint8_t  *b_ptr = (uint8_t *)h_ptr;
   *b_ptr = 0xAA;
      printf("0x%08X\n", reg.word);
   return 0;
}
```

Q 6. State the printed output of this self-modifying function dispatch system ?

```
#include <stdio.h>
typedef int (*state_fn)(int);
int state_a(int x);
int state_b(int x);
state_fn transition_table[] = {state_a, state_b};
int state_a(int x) {
   printf("A:%d ", x);
   return x > 5 ? 1 : 0;
}
int state_b(int x) {
```

MICROVISION EMBEDDED PVT LTD

```
    printf("B:%d ", x);
    return 0;
}
int main(void) {
    state_fn *curr = transition_table;
    int val = 2;
    for (int i = 0; i < 3; i++) {
        int next_idx = (*curr)(val);
        curr = transition_table + next_idx;
        val *= 3;
    }
    printf("\n");
    return 0;
}
```

Q 7. Custom embedded protocols encode flags in pointer low-bits (since heap pointers allocated on 8-byte boundaries have 3 trailing zero bits). Trace this tag-pointer program and state its output.

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#define TAG_MASK 0x07
void *set_tag(void *ptr, uint8_t tag) {
    return (void *)((uintptr_t)ptr | (tag & TAG_MASK));
}
void *clear_tag(void *ptr) {
    return (void *)((uintptr_t)ptr & ~(uintptr_t)TAG_MASK);
}
uint8_t get_tag(void *ptr) {
    return (uint8_t)((uintptr_t)ptr & TAG_MASK);
}
int main(void) {
    int *data = malloc(sizeof(int));
    *data = 42;
    void *tagged_ptr = set_tag(data, 5);
    printf("Tag: %d, ", get_tag(tagged_ptr));
```

MICROVISION EMBEDDED PVT LTD

```
    int *clean_ptr = (int *)clear_tag(tagged_ptr);
    printf("Value: %d\n", *clean_ptr);
    free(clean_ptr);
    return 0;
}
```

Q 8. What does this dynamic memory navigation code print?

```
  #include <stdio.h>
   #include <stdlib.h>
typedef struct {
   int id;
   double score;
} Student;
int main(void) {
   size_t count = 3;
   Student *pool = malloc(count * sizeof(Student));
   for (size_t i = 0; i < count; i++) {
      pool[i].id = (int)(101 + i);
      pool[i].score = 85.5 + i;
   }
   char *byte_ptr = (char *)pool;
   byte_ptr += sizeof(Student); // Move to second element
   Student *s2 = (Student *)byte_ptr;
   double *score_ptr = (double *)(byte_ptr + offsetof(Student, score));
   printf("ID: %d, Score: %.1f\n", s2->id, *score_ptr);
   free(pool);
   return 0;
}
```

Q 9. Given the code below, what is the output of matrix[0][1] and matrix[1][0] after the dynamic re-indexing operation?

```
#include <stdio.h>
#include <stdlib.h>
```

```
void swap_rows(int **mat, int r1, int r2) {
   int *tmp = mat[r1];
   mat[r1] = mat[r2];
   mat[r2] = tmp;
}
int main(void) {
   int **mat = malloc(2 * sizeof(int *));
   mat[0] = (int[]) {10, 20, 30};
   mat[1] = (int[]) {40, 50, 60};
   swap_rows(mat, 0, 1);
   int **p = mat;
   printf("%d %d\n", p[0][1], p[1][0]);
   free(mat);
   return 0;
}
```

Q 10. Look at this code carefully. Under aggressive compiler optimizations (-O3), what logical issue exists with pointer aliasing, and what can it print if the compiler reorders accesses?

```
#include <stdio.h>
void modify_val(float *f, int *i) {
   *f = 1.0f;
   *i = 0;
   // Compiler assumes *f cannot be modified by *i assignment due to strict aliasing
}
int main(void) {
   union {
      float f;
      int i;
   } u;
   modify_val(&u.f, &u.i);
   printf("f = %f\n", u.f);
   return 0;
}
```

MICROVISION EMBEDDED PVT LTD

EMBEDDED SYSTEM POINTERS, STRUCTURE, UNIONS

PART - B

MICROVISION EMBEDDED PVT LTD

Question - 01.

In ARM Cortex-M architecture, the bit-band alias region maps a 32-MB memory region so that each individual bit in a 1-MB peripheral region can be written to or read atomically using standard 32-bit pointer dereferencing.

The bit-band alias formula is:

```
Alis_Addr =  Alias_Base +  (Byte_Offset * 32 )  +  (Bit_Number  * 4)
```

Given:

```
Peripheral Bit-Band Base = 0x40000000
Peripheral Bit-Band Alias Base = 0x42000000
GPIO Output Data Register Address (GPIO_ODR) = 0x4001080C
```

Write a macro or function SET_GPIO_BIT_ATOMIC(port_odr, bit_num) that returns a volatile pointer to the bit-band alias address for bit_num (e.g., bit 5 of GPIO_ODR) and sets it to 1 without using read-modify-write (|=) operations. What is the generated alias memory address for Bit 5 of GPIO_ODR ?

Question 02.

Consider a microcontroller GPIO Peripheral mapped at base address `0x40020000`. The peripheral layout consists of:

1. `MODER` (32-bit): Mode register (2 bits per pin, Pins 0-15).
2. `OTYPER` (16-bit): Output type register (1 bit per pin). Reserved 16 bits following.
3. `ODR` (16-bit): Output data register (1 bit per pin). Reserved 16 bits following.

Construct a C `struct` overlay matching this physical register map using bit-fields for `MODER` such that `pin0_mode` occupies the lowest 2 bits. How can you set Pin 0 to Output Mode (`0b01`) and toggle Pin 0 on ODR using only structure pointers?

Question - 03.

A system clock register `RCC_PLLCFGR` at offset `0x0C` from base `0x40023800` is structured as:

Bits `[5:0]`: `PLLM` division factor (6 bits)

MICROVISION EMBEDDED PVT LTD

Bits [14:6]: PLLN multiplication factor (9 bits)

Bits [17:16]: PLLP division factor (2 bits)

Bit [22]: PLLSRC clock source select (1 bit: 0 = HSI, 1 = HSE)

Using a union containing a raw 32-bit integer and a bit-field structure, write code to configure the PLL for PLLM = 8, PLLN = 336, PLLP = 2 (encoded as 0b00), and PLLSRC = 1 (HSE) without corrupting reserved bits.

Question - 04.

A DMA controller writes 16-bit ADC samples into a circular memory buffer of size 256 samples (uint16_t buffer[256]). The DMA controller updates its hardware register DMA_NDTR (Number of Data to Transfer), which decrements from 256 down to 0 and auto-reloads back to 256.

Write a thread-safe function using pointers to calculate how many **new** samples are available for processing since the last check, returning a pointer to the start of the unread batch.

Question - 05.

In ARM Cortex-M microcontrollers, vector table relocation is achieved by writing the vector table's base memory address to the Vector Table Offset Register (VTOR, mapped at 0xE000ED08).

Write a C function that takes a dynamic RAM array containing new ISR function pointers and updates the hardware VTOR register using

pointers. What alignment rule must the new base address observe if the table contains 64 vectors ?

Question - 06.

An embedded flash memory is divided into two virtual pages for EEPROM emulation. Each page begins with a 4-byte header:

1. 0x00000000: ERASED page
2. 0x11223344: VALID page
3. 0x55667788: RECEIVE_DATA (Active migration page)

Given FLASH_PAGE0_BASE = 0x08008000 and FLASH_PAGE1_BASE = 0x0800C000, write a function using pointer casting to return a pointer to the current VALID page base address. If neither or both are valid, return NULL.

Question - 07.

A microcontroller receives an unaligned telemetry frame over UART into a raw byte buffer:

```
uint8_t rx_buf[16] = {0xAA, 0x01, 0x12, 0x34, 0x56, 0x78, 0x00, 0x00, ...};
```

Bytes [1..4] represent a 32-bit timestamp (0x78563412 Little-Endian). Dereferencing *(uint32_t *)&rx_buf[1] triggers a HardFault exception on strict architectures (e.g., ARM Cortex-M0).

Write a safe driver function using union and byte-pointer arithmetic to extract the 32-bit timestamp safely across all MCU architectures.

Question - 08.

Consider an SPI driver managing a hardware FIFO using a structure-mapped Ring Buffer:

```
typedef struct {
   uint8_t *buffer;
   uint16_t head;
   uint16_t tail;
   uint16_t capacity;
} SPI_RingBuffer_t;
```

Write a function

spi_write_bytes(SPI_RingBuffer_t *ring, const uint8_t *src, uint16_t len) that uses pointer arithmetic to copy len bytes from src into the ring buffer, correctly handling wraparound.

Question - 09.

An embedded device saves its configuration structure to internal Flash memory:

```
typedef struct {
   uint32_t baud_rate;
   uint8_t  node_id;
   uint8_t  tx_power;
   uint16_t crc16;
} __attribute__((packed)) SystemConfig_t;
```

Write a function that calculates a 16-bit XOR checksum over all fields of the configuration structure **excluding** the crc16 field itself, using byte-level pointer iteration.

Question - 10.

A system features 4 UART channels (UART0 through UART3). Each UART has a base register map defined by UART_TypeDef:

```
typedef struct {
                volatile uint32_t DR;   // Data Register

              volatile uint32_t SR;   // Status Register
                volatile uint32_t CR1;  // Control Register 1
} UART_TypeDef;
```

Define a pointer dispatch array mapping base addresses 0x4000C000, 0x4000C400, 0x4000C800, and 0x4000CC00. Write a single line function using this table to transmit byte data on channel uart_ch by waiting until the Transmit Data Register Empty (TXE, bit 7) flag is set in SR.

MICROVISION EMBEDDED PVT LTD